

Machine Learning

Session 7

Agenda:

1	Bagging
2	Boosting

Bagging

Ensemble Methods

Ensemble methods are techniques that create multiple models and then combine them to produce improved results. The key idea is that a group of models working together (an ensemble) can outperform an individual model. These methods can be divided into two main categories: **Bagging** and **Boosting**.

Bagging

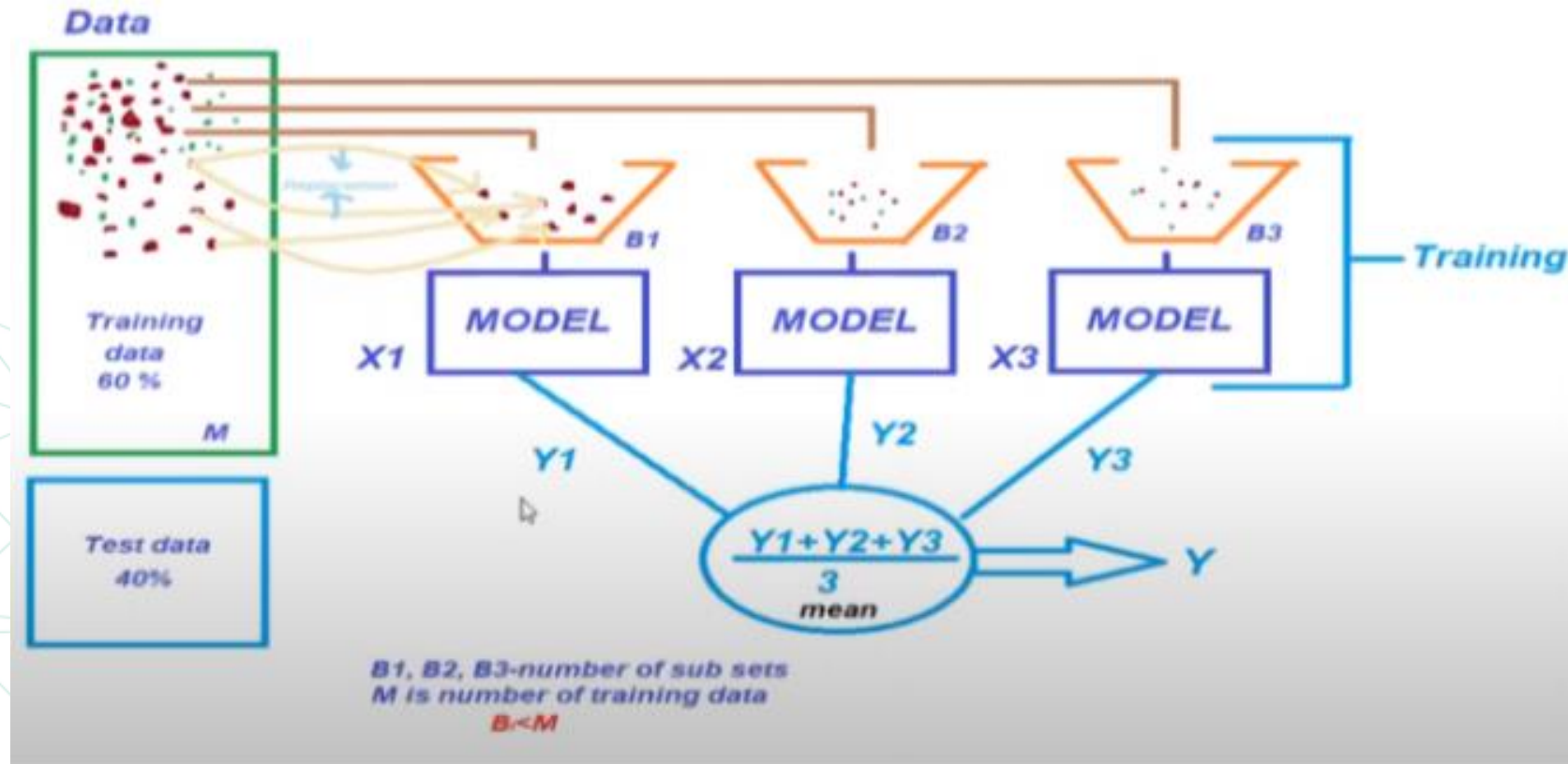
Bagging is an ensemble technique where multiple versions of a model are trained on different subsets of the training data. The predictions from these models are then combined to improve the overall performance. Bagging works particularly well with high-variance models, like decision trees, as it helps reduce overfitting.



Bagging

- Involves creating multiple subsets of data from the original dataset using sampling with replacement (each subset can have repeated samples).
- Each subset is used to train different models (e.g., Decision Trees, SVM, etc.), and their predictions are aggregated.
- For regression problems, predictions are averaged (mean), while for classification problems, the majority vote is taken (voting).
- Goal: Reduces overfitting, increases model robustness, and improves accuracy by averaging multiple models rather than relying on a single one.

Bagging



Bagging

Random Forest & Voting

Random Forest

- An extension of Bagging specifically for Decision Trees.
- Constructs multiple decision trees during training and outputs the mode of the classes (classification) or mean prediction (regression).
- Each tree votes for the class and the final prediction is made based on the majority vote.
- Helps handle overfitting and variance by using many weak learners (decision trees) to create a strong ensemble model.

Bagging

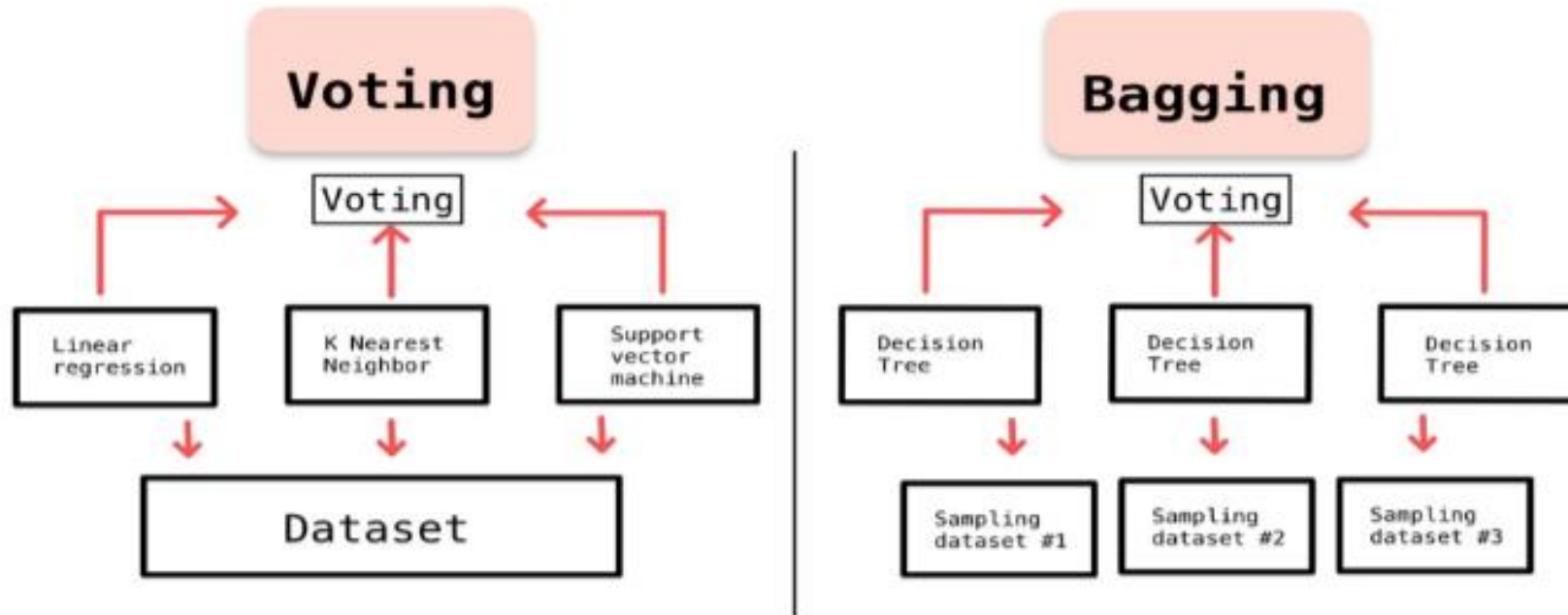
Random Forest & Voting

Voting

- For classification, we use majority voting.
- There is hard voting, where each model's prediction is considered equal, and soft voting, where each model's predicted probabilities are averaged.

Bagging

Random Forest & Voting



Bagging

Trade-offs and Performance

increasing the **number of estimators** (trees in Random Forests)

- **Pro:** Improves performance and reduces overfitting by increasing model variance.
- **Con:** Increasing estimators comes with a trade-off in computational cost and time. After a certain point, adding more estimators does not provide a significant performance boost.

Boosting

Boosting

Boosting is an ensemble technique that focuses on training weak learners sequentially, with each learner improving upon the mistakes made by the previous one. The final output is a weighted combination of the weak learners. Boosting algorithms aim to reduce bias and improve the overall performance of a model.

Boosting

BOOSTING LEARNING PROCEDURE



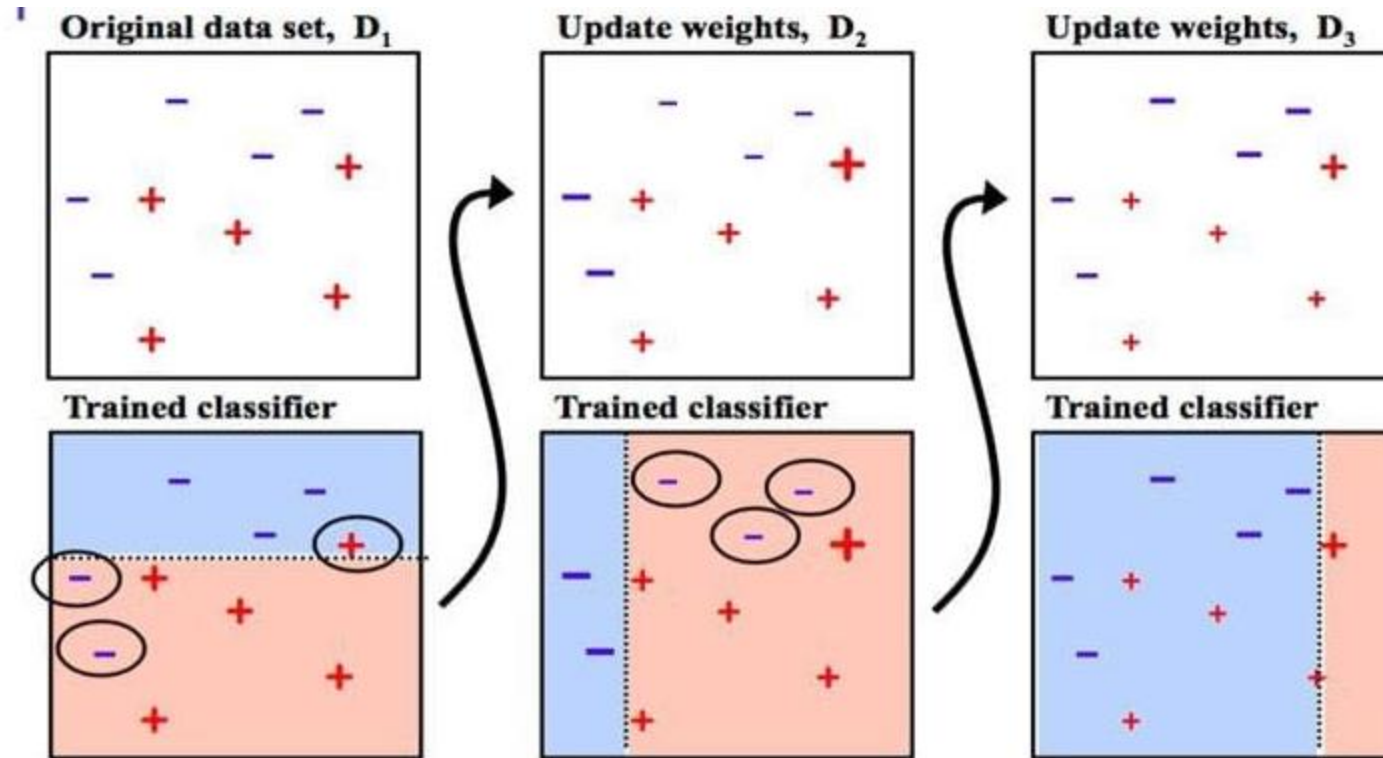
Boosting

AdaBoost (Adaptive Boosting)

AdaBoost is one of the simplest and most popular boosting algorithms. The main idea behind AdaBoost is to **combine multiple weak learners** (usually decision trees with a single split, known as "decision stumps") to form a strong classifier.

Boosting

AdaBoost (Adaptive Boosting)



Boosting

AdaBoost (Adaptive Boosting)

How it works ?

Step 1

- It starts by assigning **equal weights** to all the training data points.

Step 2

- A weak learner (like a shallow decision tree) is trained on the data. After training, **misclassified samples are given higher weights**, meaning the next model will focus more on these difficult cases.

Boosting

AdaBoost (Adaptive Boosting)

How it works ?

Step 3

- Another weak learner is trained with these updated weights, emphasizing the misclassified data from the previous learner.

Step 4

- This process repeats for a certain number of iterations or until no significant improvements are made.

Boosting

AdaBoost (Adaptive Boosting)

How it works ?

Step 5

- In the final prediction, each weak learner gets a weight based on its performance, and the results of all learners are combined (usually by a weighted majority vote).

Boosting

AdaBoost (Adaptive Boosting)

Advantages

- Focuses on difficult-to-classify samples.
- Simple to implement and works well with many weak learners.

Disadvantages

- Can be **sensitive to noise** in the data (as noisy data may get higher weights, causing the model to overfit).

Boosting

AdaBoost (Adaptive Boosting)

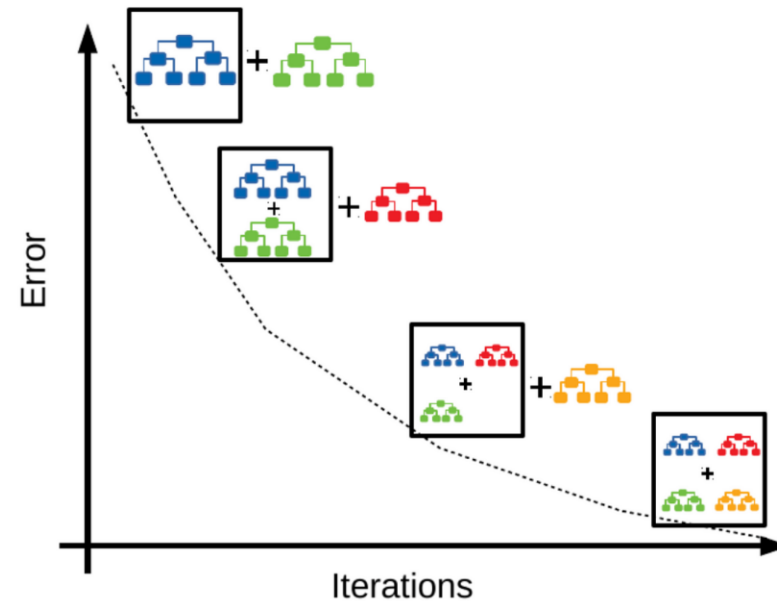
Real-life analogy

- Imagine you are teaching a class. The students who struggle (misclassified samples) receive more attention after each test (boosting round), and you gradually improve your teaching methods to help these students do better. By the end, your teaching (model) has improved for both the easy and the hard students (data).

Boosting

Gradient Boosting

Gradient Boosting takes a different approach than AdaBoost. Instead of adjusting weights for misclassified data, **Gradient Boosting builds new learners to predict the residual errors** made by the previous learners. It uses a technique called **gradient descent** to minimize errors.



Boosting

Gradient Boosting

How it works ?

Step 1

- A model (usually a weak learner like a decision tree) is trained on the dataset.

Step 2

- After the first learner makes predictions, we calculate the **errors (residuals)** between the true labels and the predicted labels.

Boosting

Gradient Boosting

How it works ?

Step 3

- The next learner is trained to **predict these residuals**, focusing on the mistakes made by the previous model.

Step 4

- This process continues, with each new learner reducing the errors of the previous one, making the model improve step by step.

Boosting

Gradient Boosting

How it works ?

Step 5

- Finally, all learners' predictions are combined to make a final prediction.

Boosting

Gradient Boosting

Advantages

- Very powerful and flexible, works well with a variety of data types.
- Minimizes both bias and variance if tuned properly.

Disadvantages

- Slow training process compared to AdaBoost.
- More complex to implement.

Boosting

Gradient Boosting

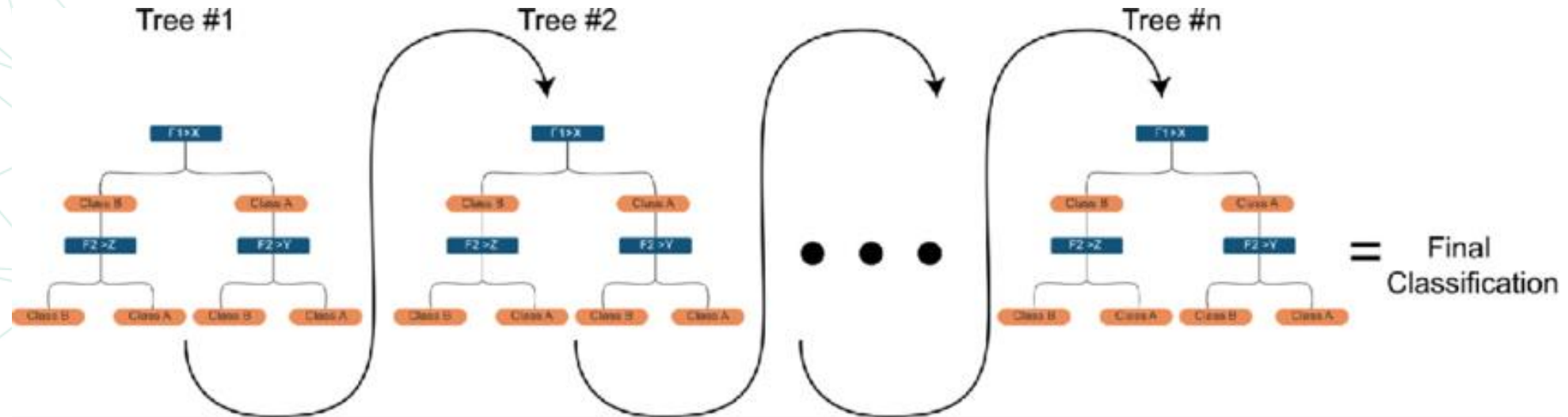
Real-life analogy

- Imagine you are building a tower. The first layer of bricks (first learner) isn't perfect, so the next layer is adjusted to fix the imperfections of the previous one (residuals). You keep building, layer by layer, fixing mistakes, until the tower (model) is strong and stable.

Boosting

XGBoost (Extreme Gradient Boosting)

XGBoost is an advanced implementation of Gradient Boosting. It's designed to be **fast, efficient, and highly scalable**. XGBoost introduces various improvements, making it one of the best performing and widely used algorithms in machine learning competitions like Kaggle.



Boosting

XGBoost (Extreme Gradient Boosting)

How it works ?

- **Optimized Gradient Boosting:** Just like Gradient Boosting, XGBoost works by sequentially adding models to predict residuals, but with several optimizations.

Boosting

XGBoost (Extreme Gradient Boosting)

Key Improvements

1. Regularization

- XGBoost applies **L1 (Lasso)** and **L2 (Ridge)** regularization to reduce overfitting.

Boosting

XGBoost (Extreme Gradient Boosting)

Key Improvements

2. Parallelized Training

- It uses **parallel and distributed computing**, which makes it faster than traditional gradient boosting.

Boosting

XGBoost (Extreme Gradient Boosting)

Key Improvements

3. Tree Pruning

- XGBoost uses **advanced tree pruning** techniques to prevent overly complex models that overfit the training data.

Boosting

XGBoost (Extreme Gradient Boosting)

Key Improvements

4. Handling Missing Data

- XGBoost has in-built methods to handle **missing data**, making it more robust.

Boosting

XGBoost (Extreme Gradient Boosting)

Key Improvements

5. Weighted Voting

- Instead of just adding models, XGBoost calculates the **optimal weight for each learner**, further improving the overall performance.

Boosting

XGBoost (Extreme Gradient Boosting)

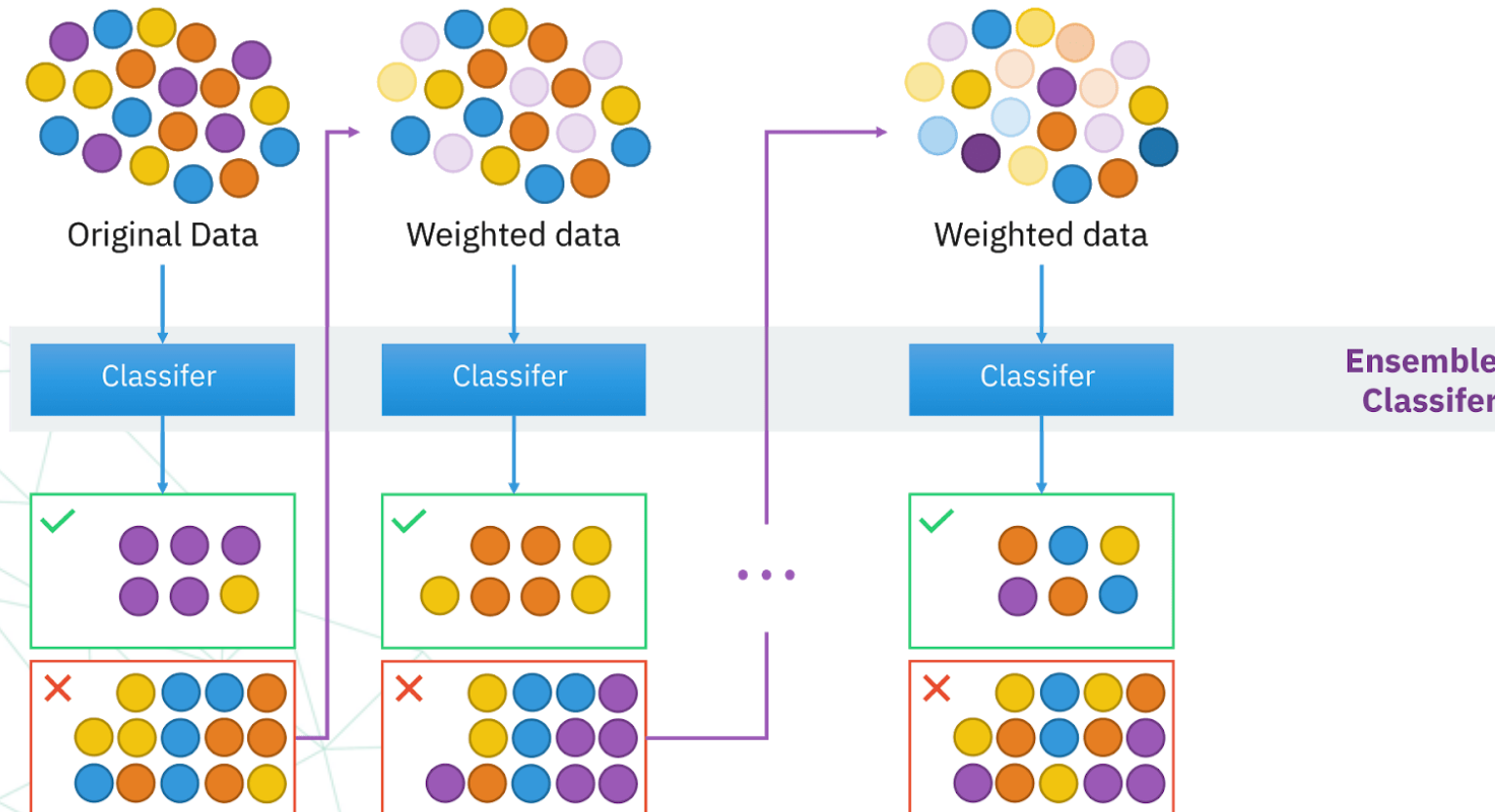
Key Improvements

6. Learning Rate

- XGBoost applies a **learning rate** (shrinkage) after each step to control how much each new model influences the overall prediction, preventing overfitting.

Boosting

XGBoost (Extreme Gradient Boosting)



Boosting

XGBoost (Extreme Gradient Boosting)

Advantages

- Fast and efficient due to its optimizations.
- Handles large datasets with many features efficiently.
- Great at reducing overfitting due to its regularization techniques.

Disadvantages

- Requires tuning a larger number of hyperparameters compared to simpler models.
- Can be complex to understand for beginners.

Boosting

XGBoost (Extreme Gradient Boosting)

Real-life analogy

- Imagine building a team of experts. With each new expert, you refine their training to correct the mistakes of the previous ones. You also ensure that these experts don't overcomplicate the task (regularization) and keep the process efficient. Each expert adds value to the final decision, making the team (model) very powerful.

Boosting

Summary of Differences

Algorithm	Main Idea	Strengths	Weaknesses
AdaBoost	Adjusts weights of misclassified data samples.	Simple, focuses on hard-to-classify samples.	Sensitive to noise, can overfit noisy data.
Gradient Boosting	Reduces errors by predicting residuals (errors) of previous models.	Very powerful, can reduce bias and variance.	Slow to train, complex to implement.
XGBoost	Optimized Gradient Boosting with faster computation and regularization.	Highly efficient, reduces overfitting, fast.	Complex, requires careful hyperparameter tuning.

Boosting

Practical Use Cases

- **AdaBoost:** Best used when you need a simple yet effective boosting technique, particularly when noise in the dataset is minimal.
- **Gradient Boosting:** Great for tabular data, where you need a highly powerful model with flexibility. Works well for both classification and regression problems.
- **XGBoost:** Preferred for large datasets and in competitions like Kaggle where speed and accuracy matter the most. It's particularly useful when overfitting is a concern due to its regularization features.

Comparison Between Bagging and Boosting

Feature	Bagging	Boosting
Training Process	Independent models (parallel)	Sequential models (each depends on the previous)
Goal	Reduce variance	Reduce bias
Focus	Equal focus on all models	Focus more on mistakes of previous models
Popular Algorithms	Random Forest	AdaBoost, Gradient Boosting, XGBoost
Speed	Faster (due to parallelization)	Slower (sequential learning)
Risk	Reduces overfitting	Can overfit if not tuned properly